

Blog Posting Pipeline — Demo

Deep Dive: everything in this repository, explained

What this document is about. This PDF documents the *demo* codebase at `blog-posting-pipeline-demo/` — a scripted, fabricated-data replay of a 7-step architecture, built by the repository owner. It does *not* document `blog-posting-pipeline`, the real, private, production content pipeline this demo describes and imitates. Every claim below is either read directly from this demo's own source files and README, or explicitly marked as an inference.

Contents

1	What this project is, in one paragraph	2
1.1	Why a demo exists at all	2
2	Core concepts a beginner needs first	2
3	The one honesty rule this whole repository is built around	3
4	Architecture: how the four files fit together	3
5	File-by-file walkthrough	3
5.1	<code>index.html</code> — page structure	3
5.2	<code>demo-data.js</code> — the data layer	3
5.3	The seven pipeline steps, in schema detail	4
5.4	<code>app.js</code> — behavior	5
6	The Publish step's post-preview, in detail	5
7	The cost figures: what's real and what's illustrative	6
8	User flows	6
8.1	Running the pipeline	6
8.2	Tab and panel relationships	6
9	How this relates to the real, private project	7
10	Challenges the owner records in the README	7
11	Image provenance	7
12	Setup, deployment, and licensing	8

13 What the owner says they'd change

8

1 What this project is, in one paragraph

`blog-posting-pipeline-demo` is a static, three-tab HTML/CSS/JavaScript page that replays the real, private `blog-posting-pipeline` project's actual 7-stage architecture (Brief, Keywords, Section plan, Writer, Image prompt, Image, Publish) against a chosen fabricated example run, for an invented brand called “Nordkast Coffee Supply.” Clicking “Run the pipeline” steps through all seven stages in order, revealing each stage's real schema shape with fabricated content. A second tab, Blog Library, mirrors a real client's actual, live blog completely: 15 real posts from `davonex.com/blog`, stored locally with real images and full text. No stage of the pipeline is actually executed and no network request is ever made; everything is pre-written data revealed on a timer.

Pipeline (in the software sense). A sequence of processing stages where each stage's output feeds the next stage's input — here, a content-generation pipeline that takes a business brief and produces a fully written, illustrated, published blog post through seven ordered steps.

1.1 Why a demo exists at all

The real system, `blog-posting-pipeline`, is a production content pipeline that researches, writes, illustrates, and publishes SEO/AEO blog posts for real Digitalise Agency clients, including `davonex.com`. Its source is the agency's live product methodology — the actual prompts, the actual per-client business rules — not just data that could be scrubbed and shared, so it stays private. This demo exists so a hiring panel or interviewer can see the real 7-step architecture actually working end to end, replayed against three invented example runs, without any of that private methodology or any real client's actual generation ever leaving the private repository.

2 Core concepts a beginner needs first

Static page, DOM, HTML/CSS/JS. See the companion *Agency Blog SaaS — Demo* explainer for full definitions; the same foundation applies here: fixed HTML/CSS/JS files, no server, no build step, with JavaScript rewriting the page's live DOM (Document Object Model) in response to clicks and timers.

JSON (JavaScript Object Notation). A simple, structured text format for representing data as nested objects, arrays, strings, and numbers — for example `{"topic": "...", "keywords": [...]}`. Several of this pipeline's steps (Brief, Keywords, Section plan) display their fabricated input/output as raw, pretty-printed JSON, exactly the shape the real system's own stages actually produce and consume internally.

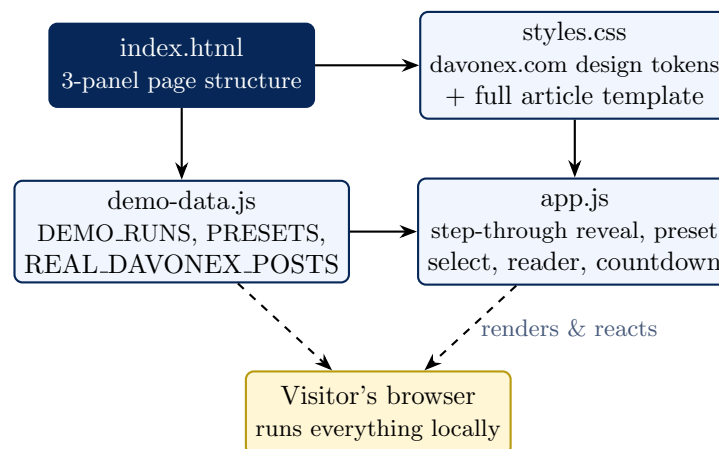
LLM (Large Language Model) / model name. Each step here is labeled with a specific AI model name (e.g. `gemini-3.1-pro-preview`), matching what the real pipeline's README says it actually uses for that stage. In this demo those labels are purely descriptive text — no model is ever called; the “output” shown was hand-written by the repository owner to match the real system's real prompt style.

IndexNow. A protocol that lets a website notify search engines immediately when a page is published or updated, instead of waiting for the engine’s crawler to discover it on its own. The real pipeline’s Publish step calls this; the demo’s Publish step shows an “IndexNow: not sent (demo)” status instead of actually calling it.

3 The one honesty rule this whole repository is built around

As with its sibling demo, everything here follows one constraint stated directly in the README: no live network calls, ever, and every fabricated number or output is visibly labeled the moment a visitor would see it. Concretely: the permanent yellow-and-black disclosure banner never disappears; every fabricated run photo is captioned “Real stock photo, not AI-generated”; the Publish step’s cost total is explicitly split into a real, anchored range versus illustrative per-step line items; and the one fabricated Blog Library card carries its own second, independent disclosure (a ribbon) so it is never confused with the 15 real posts sitting next to it.

4 Architecture: how the four files fit together



The load order in `index.html` is `styles.css`, then `demo-data.js`, then `app.js` — the same dependency pattern as the sibling demo: `app.js` reads constants that `demo-data.js` must already have defined.

5 File-by-file walkthrough

5.1 index.html — page structure

Three tabs: **Run the pipeline** (default, showing a preset-chip row, a brief preview, a Run button, a progress “rail,” and a container for the seven step cards), **Blog Library** (a grid of real and, transiently, one demo post card), and a hidden fifth section **panel-reader** for the in-page article view (there are only two visible tab buttons, plus the reader panel reached by clicking a card). The footer states plainly: “Demo build, not the production system.”

5.2 demo-data.js — the data layer

Three constants:

1. **REAL_DAVONEX_POSTS** — identical in shape and content to the sibling demo’s array of the same name: 15 real, already-published posts from `davonex.com/blog`, with full body HTML, references, and local image paths, frozen as of 2026-07-07.
2. **PRESETS** — three chip definitions, each an `{id, chip, modeledAfter}` object. `modeledAfter` is `null` for the original “Maintenance guide” example, and for the other two points at the real `davonex.com` post (title + slug) that example’s input shape was deliberately modeled after — “How to Get ChatGPT to Recommend Your E-Commerce Store” and “AEO for Designer Lighting: 2026 Organic Sales Guide,” both translated into the fictional coffee brand’s domain.
3. **DEMO_RUNS** — an object keyed by the same three preset ids, each value a full fabricated run: a `business_name`, a `topic_title`, a `hero_image` path, and a `steps` array of exactly seven step objects, described in §6 below.

5.3 The seven pipeline steps, in schema detail

Every step object shares a common envelope — `id`, `num` (e.g. "4 / 7"), `title`, `model`, and `type`, which selects how `app.js` renders that step’s body — plus type-specific fields:

#	id / type	Model label	What it shows
1	<code>brief</code> / <code>json</code>	<code>gemini-3.1-pro-preview</code>	Input (business name + description) and output (topic, description, additional details) as pretty-printed JSON
2	<code>keywords</code> / <code>json</code>	<code>gemini-3.1-pro-preview</code>	Primary keyword, keyword list, central statements, search intent, questions to answer
3	<code>plan</code> / <code>json</code>	<code>gemini-3.5-flash</code>	Meta title/description/slug plus 5 sub-topics, each with an AEO content format (table, list, etc.) and key points
4	<code>writer</code> / <code>article</code>	<code>gemini-3.1-pro-preview</code> (<code>agentic</code> , 2 tools)	Full article: intro paragraphs, headed sections with HTML body (including tables), an FAQ block, and a numbered References list
5	<code>image_prompt</code> / <code>text</code>	<code>gemini-3.5-flash</code>	A single descriptive image-generation prompt string
6	<code>image</code> / <code>image</code>	<code>gemini-3.1-flash-image-preview</code> (not called in this demo)	Preview of real stock hero photo, captioned as a stand-in, plus a usage JSON panel
7	<code>publish</code> / <code>publish</code>	<code>n/a</code>	Full post-preview (see §7), a fabricated URL, IndexNow/Sheets status, and a per-step cost table

The model label on step 4 (“agentic, 2 tools”) and the model-per-step labels throughout are asserted by the README to match the real production pipeline’s actual documented model choices — this demo’s own text explicitly frames itself as matching “the real system’s actual prompt templates, not invented from scratch.”

5.4 app.js — behavior

- `renderPresetChips()` / `selectPreset(id)` — draws the three chip buttons and, on selection, rebuilds the brief preview, the step cards, and resets the run state, so switching examples always starts clean.
- `renderBriefPreview()` — shows the selected run’s brief topic, description, and additional details in a small entity grid, plus (if applicable) a “modeled after” link to the real post that inspired this preset’s input shape.
- `buildStepsUI()` — renders all seven step cards up front (each initially hidden by CSS: `.step { display: none; }`) and a matching row of progress-rail dots, one per step title.
- `runPipeline()` — the step-through engine. It disables the Run button and all preset chips, then recursively calls a local `next()` function on a 900ms `setTimeout` loop: each call marks the previous rail dot “done,” marks the current dot and step card “active”/“visible,” smooth-scrolls to that step, and increments an index counter. When the index passes the last step, it re-enables the controls, shows a Reset button, and calls `addToLibrary(currentRunId)`.
- `renderStep(run, step)` / `renderArticle` / `renderImageStep` / `renderPostPreview` / `renderPublishStep` — a dispatcher and four specialized renderers, one per step type, each turning that step’s raw data into the right HTML shape (a JSON panel, an article layout, a photo with caption, or the full post-preview frame).
- `addToLibrary(runId)` / `demoPostCard()` / `renderLibrary()` — the same 15-minute countdown-and-expiry pattern as the sibling demo’s Blog Library, here keyed to a completed pipeline run instead of a generator submission; the demo card links back to the pipeline tab rather than opening its own reader.
- `openRealPostReader(slug)` / `switchTab(tabId)` — the same in-page reader and tab-toggle mechanics as the sibling demo.

6 The Publish step’s post-preview, in detail

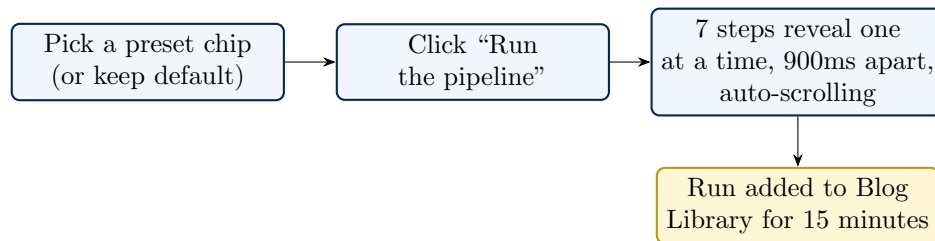
The Publish step is the most elaborate piece of UI in either demo: it builds a full, faithful recreation of `davonex.com`’s actual live article template — browser-chrome bar showing the fabricated published URL, a mini site nav using an invented “Nordkast Coffee Supply” brand mark (deliberately *not* the real client’s name or logo, to avoid impersonation), a breadcrumb, an article head, the run’s real stock hero photo, full prose with citations and a comparison table, an FAQ, a References list, an author line, and a gradient final call-to-action band — all built from the real site’s own live CSS and one real published post’s layout, populated purely with the fabricated run’s content. A closing note under the preview states explicitly: “No request is made to any real site, and the real site’s server code is not part of this demo.”

7 The cost figures: what’s real and what’s illustrative

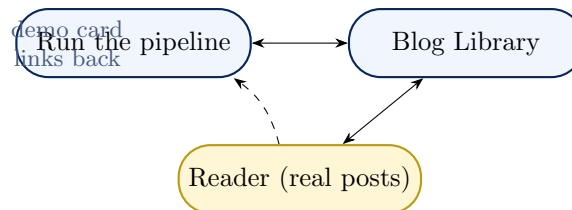
Every run’s Publish step ends with a cost table. The README is explicit about which numbers are which: the single headline total range shown per run (\$0.35–\$0.42 depending on preset) is anchored to the real, already-published production cost range stated in the real project’s own README. The per-step line items underneath that total (Brief \$0.014, Writer \$0.29, Perplexity research \$0.05, and so on) are illustrative breakdowns built for this demo and are explicitly *not* claimed to be the real system’s exact internal breakdown — a caption under the table says so directly.

8 User flows

8.1 Running the pipeline



8.2 Tab and panel relationships



Clicking the fabricated demo card’s title or “View the run” link does not open a reader at all — it jumps back to the pipeline tab, since that run’s full content already lives there as step 4’s article, not as a separate reader entry.

9 How this relates to the real, private project

Aspect	Real blog-posting-pipeline	This demo
Execution	Actually calls Gemini/Perplexity models and writes real posts for real clients	All seven steps are pre-written; nothing is generated or called
Image generation	Calls <code>gemini-3.1-flash-image-pro</code> for a real hero image	Uses one of three real, freely-licensed stock photos instead
Publish target	Publishes to real client sites, e.g. <code>davonex.com</code> , via IndexNow and a Sheets report	Shows a fabricated URL under an invented brand; IndexNow/Sheets both marked “not sent/appended (demo)”
Cost	Real per-post cost, stated as a range in the real README	Same headline range reused verbatim; per-step breakdown is illustrative only

10 Challenges the owner records in the README

- Making fabricated output feel like the real system’s voice — solved by matching the Writer step’s structure (answer-first intro, direct-answer section openings, comparison table, FAQ block, numbered citations with a References list) to the real system’s actual prompt templates for all three runs, rather than inventing generic marketing copy.
- Deciding what’s honestly reusable from a real client’s public content — resolved by reproducing `davonex.com`’s real, live, already-public post list, images, and text locally with provenance links, rather than inventing a fake library or leaving it as titles-only stubs.
- Sourcing real content without breaking the self-contained, no-live-calls rule — the 15 real posts’ images and text are fetched once and stored under `assets/posts/`; the three fabricated-run photos use a separate, freely-licensed stock-photo sourcing approach (CC BY / CC BY-SA / public domain) since they illustrate an invented brand rather than reproducing a real client’s own work.
- Disclosing fabrication without undermining the demo — solved with a pinned, non-dismissable banner visible through the whole page, plus the Library’s demo card carrying its own second, independent disclosure ribbon.

11 Image provenance

`assets/images/CREDITS.md` documents the three stock photos used for the fabricated runs, each under a separate, named open license distinct from this repository’s own PolyForm Strict License:

File	Source		Author	License
espresso-machine.webp	Wikimedia mons	Com-	HungryHuy	CC BY 2.0
coffee-beans.webp	Wikimedia mons	Com-	MarkSweep	Public domain
coffee-cherries.webp	Wikimedia mons	Com-	Luai Al-Mamari	CC BY-SA 4.0

Per CC BY-SA’s share-alike term, the cropped `coffee-cherries.webp` file specifically remains under CC BY-SA 4.0 rather than the repository’s own license. Separately, `assets/posts/CREDITS.md` documents that the 15 real `davonex.com` post images and bodies are the owner’s own agency’s real client work, not third-party stock, already disclosed elsewhere in the portfolio.

12 Setup, deployment, and licensing

Setup: none — open `index.html` or serve the folder statically. **Deployment:** `netlify.toml` sets `[build] publish = "."`, serving the repository root with no build command. **License:** PolyForm Strict License 1.0.0, copyright Salman Adnan, 2026, covering this repo’s original code and fabricated content (not the separately-licensed stock photos or the real, disclosed client content).

13 What the owner says they’d change

The README’s “What I’d do differently” notes that a real client publishes dozens of posts a month, so this static 15-post snapshot will drift out of date; a live version could re-fetch that list periodically, but that exception was deliberately skipped to keep the “no live calls” guarantee airtight.